FUNDAMENTALS OF SOFTWARE ENGINEERING

Final Assignment

Overview

This final assignment evaluates your team's complete term project — both the engineering practices applied during the implementation phase (Weeks 9–13) and the working product itself. The assignment has three components: a comprehensive Final Project Report, a live Final Presentation, and a Live Demonstration of your deployed application.

The midterm assessed your design and planning. This final assessment evaluates what you actually built and how you built it. As with the midterm, all submitted work must be original. You may not copy and paste from previous lab submissions or your midterm report. Where material overlaps with earlier work, it must be revised, refined, and consolidated into a polished final form.

Part 1: Final Project Report

Submit a single, well-structured report that documents the implementation, testing, deployment, and quality assurance of your project. Each section must reflect the actual state of your project at submission time — not aspirational plans.

1.1 Executive Summary & Project Recap

A concise overview recapping the project goal and summarising what was built. If anything significant changed since the midterm — scope, architecture, or technology choices — document it here with justification.

1.2 Final Architecture & Implementation Notes

Provide updated architecture diagram(s) reflecting what was actually implemented. Note any deviations from the midterm design and explain the reasoning. Include a brief description of the key modules, components, or services and their responsibilities.

1.3 Clean Code & Code Review Practices

Demonstrate the application of Clean Code principles in your codebase. Include:

- Examples drawn from your codebase, each with a short explanation of what improved (e.g. naming, function size, reduced nesting, removal of magic numbers).
- A summary of team code review process: how feedback was incorporated, and at least one example pull request with its review comments.

1.4 Testing Strategy & Coverage

Describe your testing approach across the testing pyramid (unit, integration, end-to-end where applicable). Include:

- A summary of what is tested and what is not, with reasoning.

- The final coverage report (target: 70% minimum on core modules) — include the actual screenshot or output of the report.

1.5 CI/CD Pipeline

Document your continuous integration and deployment pipeline. Include:

- The pipeline configuration file (or a clearly readable excerpt).
- Annotated screenshots of recent successful and failed pipeline runs.

1.6 Containerization & Live Deployment

Document the deployment of your application:

- The Dockerfile and, where applicable, the docker-compose.yml.
- The cloud platform used and the reasoning for the choice.
- **The public URL where the deployed application can be reached.**
- How environment variables and secrets are managed — secrets must NOT be present in the repository.

1.7 Code Quality, Security & Technical Debt

- The static analysis tool integrated into your pipeline, with a list of issues identified, fixed, and remaining.
- A “Known Issues & Technical Debt” appendix documenting items that were consciously deferred and why.

1.8 GitHub Repository

- Final repository URL.
- README.md including: project description, tech stack, local setup instructions, deployment URL, and team members.
- A brief commit and branching summary illustrating the implementation arc since the midterm.

1.9 Sprint Closure & Jira

Include annotated, high-resolution screenshots of:

- All sprints completed since the midterm, showing planned versus delivered work.
- The final state of the product backlog, indicating what shipped and what was deferred.

1.10 Team Contributions & Reflection

- A short per-member contribution summary (one to two sentences each is sufficient).
- A team reflection: what worked well, what you would do differently, and the most valuable lesson learned.

Part 2: Final Presentation & Live Demo

Each team will deliver a live, in-class presentation followed by a working demonstration of the deployed application. The demo is a mandatory component — a non-deployed project cannot pass this assignment.

Presentation Requirements

Requirement	Detail
Duration	15 minutes total — approximately 10 minutes of slides plus a 5-minute live demo
Language	English (mandatory)
Participation	All team members must present a portion of the slides AND participate in the demo
Deployment	The application MUST be reachable at a public URL during the presentation
Slides	Submitted as a file on Moodle (PDF or PPTX)

Suggested Slide Structure

Teams may organise their slides as they see fit, but the following structure is recommended to ensure full coverage within the time limit.

Slide	Suggested content
1	Project recap — name, team, what the product does (very brief)
2	What changed since the midterm — scope, design, or architectural pivots
3	Final architecture — one clean diagram of what was actually built
4–5	Engineering practices — clean code, code review, testing strategy and coverage
6	CI/CD pipeline — show pipeline runs and the gates protecting the main branch
7	Deployment setup — containerization, cloud platform, environment management
8	Quality & security — static analysis findings, technical debt
9	Live demo (≈5 minutes) — walk through the deployed app's main user stories
10	Lessons learned, team contributions, and future work

Live Demo Requirements

- The application must be accessed at its public deployment URL during the demo — not from a localhost server.
- Walk through at least one of the user stories that were submitted in the midterm report. If any of those stories changed, briefly note the change.
- All team members should be ready to answer questions about how the deployed instance is built, tested, and deployed.

Deliverables

Submit all files via Moodle before the deadline. Late submissions will not be accepted unless prior approval has been obtained from the instructor.

Deliverable	File format
Part 1: Final Project Report	PDF
Part 2: Presentation Slides	PDF or PPTX

Naming convention: TeamXX_FinalReport.pdf and TeamXX_FinalPresentation.pptx (replace XX with your team identifier).

Grading Criteria

The final assignment will be assessed using the following rubric:

Criterion	Focus area	Weight
CI/CD	Functional automated pipeline with appropriate test, lint, and merge gates	10%
Report Quality	Completeness, originality, coherence as a single document	40%
Presentation Clarity	Communication, all members presenting, English used throughout	40%
Live Demo	Clear walkthrough of the deployed product and its user stories	10%

Academic Integrity

This is a team assignment. All work submitted must be the original work of the team. Plagiarism, copying from external sources without attribution, or submitting earlier lab or midterm material without meaningful revision will result in a grade of zero for the affected team members and may be subject to further academic disciplinary action.

All team members are expected to contribute equally to both the implementation work and the final deliverables. The instructor may conduct individual interviews or commit-history reviews following the presentation if there are concerns about unequal contribution.

Important Notes

- The deployed application MUST be live during your presentation slot. A non-deployed project cannot pass this assignment.
- All diagrams and screenshots must be high-resolution and legible — blurry or unreadable artefacts will not receive full marks.
- The report should read as a coherent, professionally formatted document — not a stitched-together collection of lab outputs.
- All team members must actively present AND take part in the live demo. Teams where one or more members do not speak will lose marks.
- Slides and the report must be uploaded to Moodle before the presentation session begins.
- Q&A may follow each presentation at the instructor's discretion; all members should be prepared to answer questions about any part of the system.

Questions

If you have questions about the assignment, please post them in the course discussion forum on Moodle so that all students can benefit from the answers. For personal matters, contact your instructor directly via the university email system.